

# **harness-engineering**

**A hands-on course in building agents by building their harness**

Chase Dovey · Average Joes Lab

## Three parts

1. **The course.** What a harness is made of, then ten lessons that build one
2. **The paper.** The course's thesis, tested against Claude Code, OpenAI Codex and opencode
3. **quark at work.** The harness you build in Lesson 4 doing real work on this repo

# No definition

- Framework, scaffold, runtime, orchestration layer: different words, same thing
- People are bad at saying what they mean and good at showing it
- A definition only says what something is; **a mechanism shows it**
- So instead of saying what I mean, I'll show you

# An agent works by two mechanistic primitives

- The model predicts: **TokensOut = Model(TokensIn)**
- The harness does everything else

**Agent = Harness(Model)**

# This course is about the harness

- The model is built by **model development**: training data, compute, and the methods to train and evaluate a model
- How a model works inside is an optional deep dive
- The harness is the part you build here

# What is a harness made of?

| Primitive              | What it does                                                            |
|------------------------|-------------------------------------------------------------------------|
| <b>control flow</b>    | how information flows between the other four                            |
| <b>input</b>           | how inputs are gathered, from a person or the world                     |
| <b>context</b>         | what the request holds, and how inputs are presented in it              |
| <b>model interface</b> | how the harness interfaces with the model                               |
| <b>output</b>          | how the model's outputs are handled: shown to a person, or run as tools |

## Same five primitives, different choices

- Run the other four once: **a single call**
- Loop and hand back after every reply: **a chatbot**
- Keep looping while the model asks to use tools: **an agent**
- Claude Code, Cursor, Codex, and the harness you'll build here are all these five primitives with different choices made

# The method

- Take a thing and ask: *what is it, and by what mechanistic primitives does it work?*
- An agent is a model and a harness
- A harness is control flow, input, context, model interface and output
- Ask once more and the answers stop being shared: one harness reads a terminal, another a Slack channel
- That's where taking apart ends



## Then build it back up

- One primitive at a time, outward from the model
- Lesson 1 calls the model and nothing else
- Lessons 2–4 each add what the last one was missing: input and output, then control flow, then context
- Each one's `quark.py` is the previous lesson's plus that

# quark

- The example throughout is **quark**, my own agent
- One way to build each primitive, not the only way
- Every lesson shows:
  - i. its idea on its own, in a small file you can run
  - ii. what quark adds to it and why, with runs
  - iii. the other things it could be, in words

# The primitives: Lessons 1–4

| # | Lesson           | You build                                            |
|---|------------------|------------------------------------------------------|
| 1 | Model interface  | one call, and its reply as it streams back           |
| 2 | Input and output | how inputs are gathered, and how outputs are handled |
| 3 | Control flow     | the loop that sends a result back and goes again     |
| 4 | Context          | who it is, what's happened, what it remembers        |

# A harness that works isn't yet one you'd run unattended

- The production layers add **hardening**: safe, watchable, dependable and fast
- None of it is a new primitive
- Each layer folds back into the primitives it's built on, and each lesson shows where
- Each lesson's `quark.py` is the one before it plus that layer, and nothing else

# The production layers: Lessons 5–10

| #  | Lesson        | Built on                                       |
|----|---------------|------------------------------------------------|
| 5  | Sandboxing    | output                                         |
| 6  | Guardrails    | control flow                                   |
| 7  | Observability | control flow                                   |
| 8  | Resilience    | model interface, output                        |
| 9  | Performance   | context, model interface, output, control flow |
| 10 | Evaluation    | the whole harness                              |

## Jev: a second model

- From Lesson 5 on, each layer's concept comes twice: on its own, then with one question for **Jev**
- Jev is a small model that makes decisions instead of writing text
- What a command needs from the box, whether it only reads, whether a tool failed, whether a failure will pass, how big a request is, whether the agent finished
- Then quark gets both

# The paper

## ***Agent = Harness(Model): Five Mechanistic Primitives of LLM Agent Harnesses***

- **The claim:** an agent is a model wrapped in a harness, and every harness is made of five primitives
- **Production:** every production concern folds back into those five instead of adding a sixth
- **A test on real harnesses:** Claude Code, OpenAI Codex and opencode, part by part
- **A case study:** the ten runs of quark at work, failures included

## quark at work

The `quark.py` you finish Lesson 4 with, nothing added, doing real work on this repo. In ten sessions, it:

- brought the four primitive lessons' slide decks into line with the lessons
- turned all ten decks into PDFs
- reviewed the paper
- weighed in on the order of the production lessons



# Memory at work

- *"Now do the same for Lesson N's slides."*
- A new session has no idea what "the same" means
- The skill was in the index in its prompt, so its first command was to read it
- Later it answered questions about its past from its episodes

## What we checked

- We reviewed everything quark changed before it was merged
- Its corrections to the paper were right
- We fixed two unreadable slides by hand
- Its memory slipped: a fact it didn't replace, a note where a skill edit belonged

# Setup

- macOS or Linux · Python 3.13+ · uv · an Anthropic API key
- Lessons 5–10 also need Docker, and a TypeSafe key for Jev
- Put your keys in `.env` at the root of the repo (it's gitignored)

```
cp .env.example .env          # then add your keys to .env
export UV_ENV_FILE=.env
```

## A warning

From Lesson 2 on, the agent runs shell commands the model writes, with no confirmation.

**Run it somewhere you can afford to lose.**

# **Start with Lesson 1**

**Model interface**